

Привет!

Отправляем тебе на выбор **3** задания. Ты можешь выполнить любое из них. Желаем удачи!

Задание 1

Необходимо создать приложение «Заметки» на одном из следующих языков программирования.

- **SQL (T-SQL);**
- **Delphi.**

Приложение может быть сделано как в консольном виде, так и в браузере или в виде приложения на Android, iOS. Строгих требований к реализации и функциональности нет, подойди к заданию творчески! Главное для нас — посмотреть, как ты выполнишь задачу и какие инструменты в разработке используешь.

Обязательно:

1. Создание одной простейшей заметки только с текстом.
2. Редактирование заметки в окне собственного приложения.
3. Сохранение заметки между сеансами приложения, в любом формате.
4. При первом запуске в приложении должна быть одна заметка с текстом.

Желательно:

1. Возможность создания нескольких заметок в приложении.
2. Вывод списка существующих заметок.
3. Возможность редактирования любой заметки из списка.
4. Удаление заметок.

Идеи для улучшения:

1. Возможность выделять текст курсивом, жирным и прочее.
2. Возможность менять шрифт и размер текста.
3. Вставка картинок.

Удели внимание качеству кода, а также стилистическому оформлению. Учитывай, что твой продукт может стать популярным, и над исходным кодом будут работать другие разработчики. Удачи!

Задание 2

Тебе необходимо создать backend для работы со складом рулонов металла. Это задание нужно выполнить на **Python**.

Рулон: id, длина, вес, дата добавления, дата удаления.

Обязательные пункты:

1. RESTFull API:

- a. добавление нового рулона на склад. Длина и вес — обязательные параметры. В случае успеха возвращает добавленный рулон;
- b. удаление рулона с указанным id со склада. В случае успеха возвращает удалённый рулон;
- c. получение списка рулонов со склада. Рассмотреть возможность фильтрации по одному из диапазонов единовременно (id/веса/длины/даты добавления/даты удаления со склада);
- d. получение статистики по рулонам за определённый период:
 - количество добавленных рулонов;
 - количество удалённых рулонов;
 - средняя длина, вес рулонов, находившихся на складе в этот период;
 - максимальная и минимальная длина и вес рулонов, находившихся на складе в этот период;
 - суммарный вес рулонов на складе за период;
 - максимальный и минимальный промежуток между добавлением и удалением рулона.

2. Данные по рулонам должны храниться в базе данных (желательно PostgreSQL/SQLite).
3. Должны быть обработаны стандартные кейсы ошибок (например, недоступна БД, не существует рулон при какой-то работе с ним).
4. Используемый стек: FastAPI, SQLAlchemy, pydantic (версии и до 2.0, и после 2.0 подойдут).

Бонусные баллы:

1. Получение списка рулонов с фильтрацией (п.1, с.) работает по комбинации нескольких диапазонов сразу.
2. Получение статистики по рулонам (п.1, d.) дополнительно возвращает:
 - день, когда на складе находилось минимальное и максимальное количество рулонов за указанный период;
 - день, когда суммарный вес рулонов на складе был минимальным и максимальным в указанный период.
3. Проект должен быть обернут в Docker.
4. Конфигурации к подключению к БД должны быть настраиваемыми через файл или ENV.
5. Проект должен быть покрыт тестами.
6. Проект должен проходить туру, flake8 и прочее.

7. Отсутствие глобальных переменных.

Идеи для улучшения:

1. Использование абстракций/интерфейсов для возможной замены транспорта. Например, с PostgreSQL на какое-нибудь хранилище (InMemory / Redis / File).
2. Тесты не должны зависеть от реальной базы данных (т.е. использовать класс, фальшивку или моки).

Нам очень интересно посмотреть, как ты программируешь. Если что-то не удалось реализовать — мы смотрим образ мышления, а не конечный результат.

Задание 3

Кейс: имеется 3 поставщика, каждый из поставщиков может поставлять 2 вида груш и 2 вида яблок. Поставщики заранее сообщают свои цены на виды продукции на определенный период поставок.

Задача:

1. Создать интерфейс приёмки поставок от поставщиков. В одной поставке от поставщика может быть несколько видов продукции.
2. Создать отчёт. За выбранный период показать поступление видов продукции по поставщикам с итогами по весу и стоимости.

Требования:

1. Данные приложения должны сохраняться в БД путем формирования таблиц из объектов Backend (СУБД любая).
2. Backend – Hibernate, Java, Spring Framework.
3. Frontend – любая реализация (Javascript, Java мы не будем рассматривать эту часть).
4. Нам важно получить исходные коды программы + работающее мини приложение. Выложить все файлы в GitHub или GitLab.

Мы знаем, что у тебя всё получится! Главное – показать все свои способности и умения в этом задании☺